

Sprint 6: Purchase Reservation and Settlement

How an accepted quote becomes safely allocated crypto backing

Completed and production migration verified on 2 September 2026

Honest status: Delivered. The schema, domain service, API, worker, atomic repository, tests, and canonical documentation are implemented. Local quality gates and the real PostgreSQL purchase lifecycle scenario pass. Migration 11 is applied to Railway production, and the temporary database proxy used for verification was deleted.

1. Goal

Consume one unexpired purchase quote after Sendaza locks the exact fiat debit, reserve the quote's crypto amount on its configured asset-network, and safely converge with Sendaza's ledger result. SLE does not hold customer fiat and never writes Sendaza balances.

2. Connection to Earlier Sprints

Sprint 4 created immutable quotes containing exact debit, credit, spread, fee, precision, expiry, and backing asset-network evidence. Sprint 5 created fresh, independently matched treasury inventory for each asset-network. Sprint 6 joins those results: a valid quote tells SLE what to reserve, and fresh inventory proves the reservation can be backed.

3. What Was Built

Domain: `packages/domain/src/purchase.ts` defines the purchase states, stable failures, service boundary, response view, and leased timeout batch.

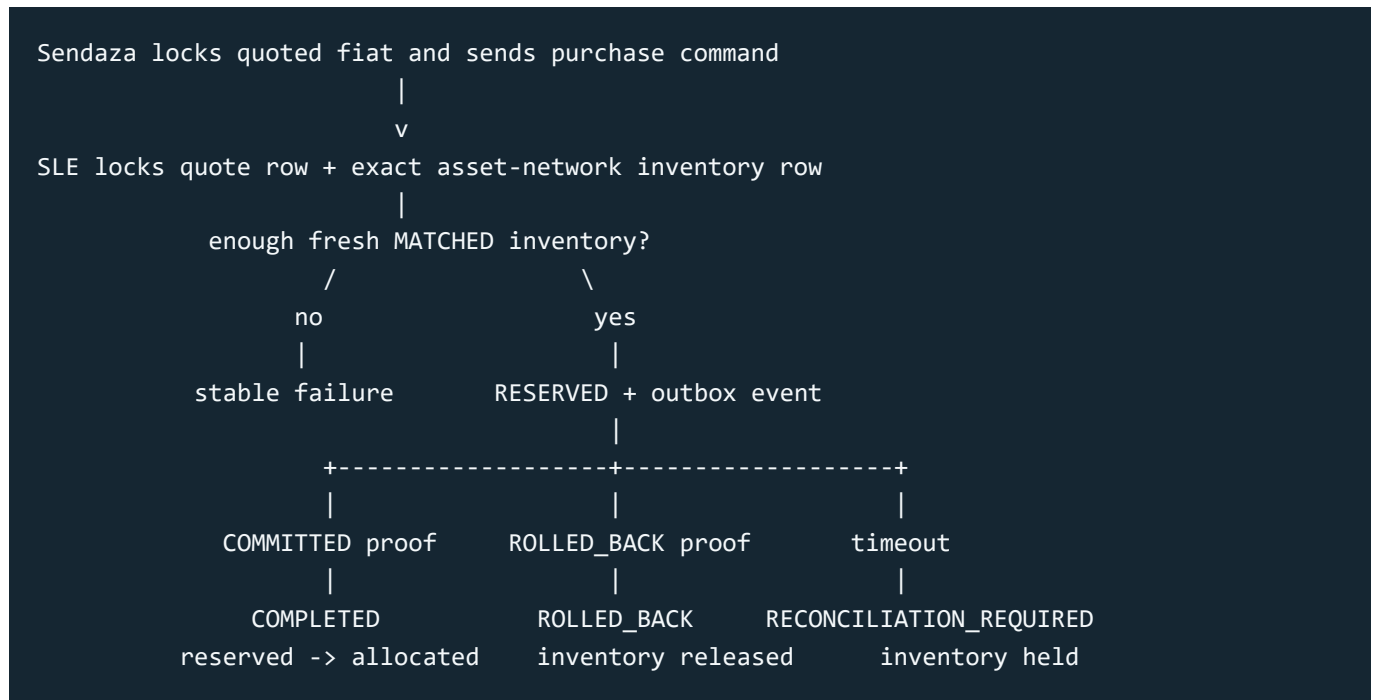
Database: `prisma-purchase.repository.ts` serializes competing reservations with row locks and commits purchase, reservation, inventory, transition, timeout job, and outbox event together. The forward migration adds purchase history, immutable settlement evidence, timeout jobs, and separately tracked allocated inventory.

API: `POST /purchases`, `POST /purchases/:id/settlement`, and `GET /purchases/:id` expose a strict Sendaza-only contract. Clients cannot select wallets, providers, networks, policies, amounts, or fees.

Worker: the purchase timeout worker leases bounded batches. A missing acknowledgement moves a reservation to reconciliation and keeps its inventory held.

4. End-to-End Example

Suppose a configured quote says the customer pays `200000.00` fiat and receives `6250000.000000` units of a configured token on one network. Sendaza first locks exactly `200000.00` in its own ledger, then sends the quote ID and its lock reference to SLE.



Two concurrent purchases cannot spend the same capacity because PostgreSQL locks the inventory row. A successful settlement converts reserved units into allocated units; they remain unavailable because they now back a customer liability.

5. Fit in the Complete System

Sendaza remains responsible for authentication, KYC, customer authorization, fiat locks, double-entry journals, balances, and the customer interface. SLE remains responsible for quote economics and treasury backing. Fireblocks MPC is not called during a purchase because no blockchain transfer occurs; custody is relevant later when an external withdrawal moves allocated crypto on-chain.

6. Rules Protected

- All amounts are atomic-unit `bigint` values internally and decimal strings at the API.
- A quote is consumed once and a financial mutation is idempotency protected.
- Inventory is scoped by asset-network; the same token on two networks is not interchangeable.
- Reservation cannot make sellable inventory negative.
- State changes and their outbound event share one PostgreSQL transaction.
- Timeout is uncertainty, not proof of failure, so inventory stays held.
- Completed and rolled-back history is immutable.

- No private key, seed phrase, or customer ledger mutation enters SLE.

7. Test and Environment Evidence

Passed: ESLint; strict TypeScript compilation; API and worker production builds; 42 unit suites with 233 tests, including purchase calculations and states, parser rejection, API mappings, and treasury refresh preservation.

Passed on real PostgreSQL: 11 integration cases passed across the final validation runs. The Sprint 6 scenario independently raced two purchases against limited inventory, proved that only one succeeded, committed one reservation, released a proven rollback, held a timed-out reservation, and verified lifecycle outbox events. The Railway proxy was slow, so the runner used a longer command-line timeout without changing assertions.

Production evidence: Prisma reported migration 11 pending, applied `20260902000300_purchase_reservations`, and then reported all 11 migrations current. The temporary public TCP proxy was deleted immediately; Railway then reported zero proxies.

8. Limitations and Next Dependency

The generic HTTP idempotency record and the purchase transaction still commit separately. Database uniqueness prevents duplicate allocation, but a crash after purchase commit and before response persistence can leave an idempotency key in progress. Automated recovery is required before real-funds launch.

Sprint 7 now builds cached network-fee snapshots for external withdrawals; it does not change purchase pricing or reservation behavior.

9. Glossary

Reservation: crypto inventory temporarily held for one purchase. **Allocation:** treasury holdings committed as backing for a customer crypto liability. **Asset-network:** one asset on one blockchain, such as USDT on Ethereum versus USDT on Tron. **Row lock:** a database rule that makes competing updates wait their turn. **Outbox:** events stored with the financial change and delivered later. **Reconciliation:** investigation of an outcome that cannot be safely classified. **Compensating record:** a new linked correction instead of rewriting history.

Sprint 6 learning and handover report. Canonical behavior remains in the architecture, API, ledger integration, design decision, and module documents.